

Universidad de San Carlos de Guatemala

Centro Universitario de Occidente CUNOC

División de las Ciencias de la Ingeniería

Organización de Lenguajes y Compiladores 2

Sección: A

Docente: Mario Moisés Ramírez Tobar



“Manual de Usuario Practica 1”

Estudiante:

Mynor Miguel Monzón Martínez 202230884

Quetzaltenango, 24 de agosto de

2026

Manual de Usuario Codex Latinus

Introduccion

Este manual describe el uso del Compilador Codex Latinus, una aplicacion de escritorio desarrollada en Java que permite escribir, editar y analizar codigo en el lenguaje Codex Latinus directamente desde una interfaz grafica.

Codex Latinus es un lenguaje de programacion de sintaxis inspirada en el latin, disenado con fines educativos para el curso de Organizacion de Lenguajes y Compiladores 2. El lenguaje cuenta con cinco tipos de datos, arreglos, estructuras anidables, funciones con y sin retorno, tres tipos de ciclos, condicionales encadenados y funciones especiales de entrada y salida.

La aplicacion permite abrir y guardar archivos con extension .lat, analizar el codigo escrito en sus tres fases, visualizar los resultados en una consola integrada y consultar cuatro reportes: el reporte de errores, la grafica del Arbol de Sintaxis Abstracta, la grafica de la tabla de simbolos y la simulacion paso a paso de la pila de analisis. Adicionalmente permite traducir el codigo al idioma PigLatin y descargarlo con extension .pig.

Es importante aclarar un aspecto del alcance de la herramienta: el compilador analiza y traduce el codigo, pero no lo ejecuta. Las funciones especiales de impresion y lectura se validan y se traducen, pero no producen salida ni solicitan datos al usuario durante el analisis.

1. Requisitos del Sistema

Para ejecutar el Compilador Codex Latinus se necesita lo siguiente:

- Sistema operativo: Ubuntu Linux, Windows o macOS.
- Java Runtime Environment version 21 o superior.
- Graphviz instalado en el sistema para generar los reportes graficos como imagen. En Ubuntu se instala con el comando: `sudo apt install graphviz`
- Si Graphviz no esta disponible, la aplicacion sigue funcionando y ofrece copiar el codigo del grafico para renderizarlo en un servicio en linea.

2. Instalacion y Ejecucion

2.1 Obtener el programa

El programa se puede obtener de dos formas:

- Clonando el repositorio de GitHub del proyecto y compilandolo con Maven.
- Descargando el archivo JAR ejecutable publicado en la seccion de Releases del repositorio.

2.2 Compilar desde el código fuente

Desde la carpeta raíz del proyecto, ejecutar el siguiente comando en una terminal:

```
mvn clean compile
```

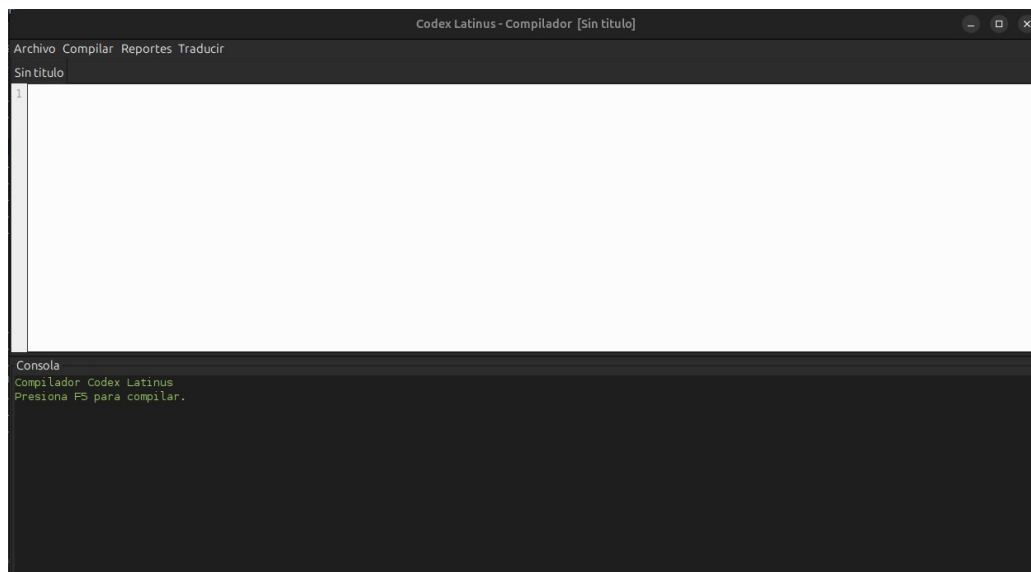
Este comando descarga las dependencias, genera el analizador a partir de la gramática y compila todo el código fuente. Al finalizar debe mostrar el mensaje BUILD SUCCESS.

2.3 Ejecutar el programa

Para iniciar la aplicación desde Maven:

```
mvn exec:java -Dexec.mainClass=com.mycompany.\
practica_1_compi_2_mynor_monzon_1s_2026.\
Practica_1_Compi_2_Mynor_Monzon_1S_2026
```

También se puede ejecutar desde el entorno NetBeans presionando el botón de ejecución del proyecto. Al iniciar, la ventana principal se abre automáticamente.



3. Interfaz Principal

La ventana principal está dividida en las siguientes áreas:

Area	Descripción
Editor de código	Área principal donde se escribe o se pega el código. Utiliza fuente monoespaciada, muestra números de línea y aplica coloreado de sintaxis en tiempo real. Soporta varios archivos abiertos en pestañas simultáneas.
Números de línea	Columna a la izquierda del editor que se actualiza automáticamente conforme se escribe.

Area	Descripcion
Consola de salida	Panel inferior donde aparecen los mensajes del compilador y el listado de errores encontrados.
Barra de estado	Franja inferior que muestra la ruta del archivo abierto y la linea y columna donde se encuentra el cursor.
Barra de menu	Contiene los menus Archivo, Compilar, Reportes y Traducir.

3.1 Coloreado de sintaxis

El editor colorea el codigo automaticamente conforme se escribe, sin necesidad de compilar. Los colores utilizados son los siguientes:

Elemento	Color
Marcadores de seccion	Morado en negrita
Palabras reservadas	Azul en negrita
Tipos de datos	Turquesa en negrita
Cadenas y caracteres	Verde
Numeros y valores booleanos	Naranja
Operadores	Rojo oscuro
Comentarios	Gris en cursiva

```

Archivo  Compilar  Reportes  Traducir
Sin título: basico.lat
15  ## =====
16  SECCION DE VARIABLES GLOBALES
17  Aquí van variables, arreglos y estructuras. Es la única sección
18  donde se pueden declarar, junto al bloque VARIABLES[ ] local
19  de cada función.
20  ===== ##
21  VARIABLES>
22
23  ## -----
24  1. TIPOS PRIMITIVOS
25  Los cinco tipos del lenguaje con valor inicial.
26  Verificar en la tabla de símbolos: 5 variables, rol Variable,
27  ámbito global, con el tipo correcto cada una.
28  ----- ##
29  esto v_entero      : numerus 42;
30  esto v_decimal     : decimalis 9.81;
31  esto v_texto       : textum "Codex Latinus";
32  esto v_letra       : littera 'a';
33  esto v_booleano    : bool verum;
34
35  ## -----
36  2. BOOLEANOS EN SUS DOS FORMAS
37  El foro confirmó que conviven la forma tipada y la especial.
38  Ambas deben producir un símbolo de tipo bool.
39  ----- ##
40  esto b_tipado_v    : bool verum;
41  esto b_tipado_f    : bool falsus;

```

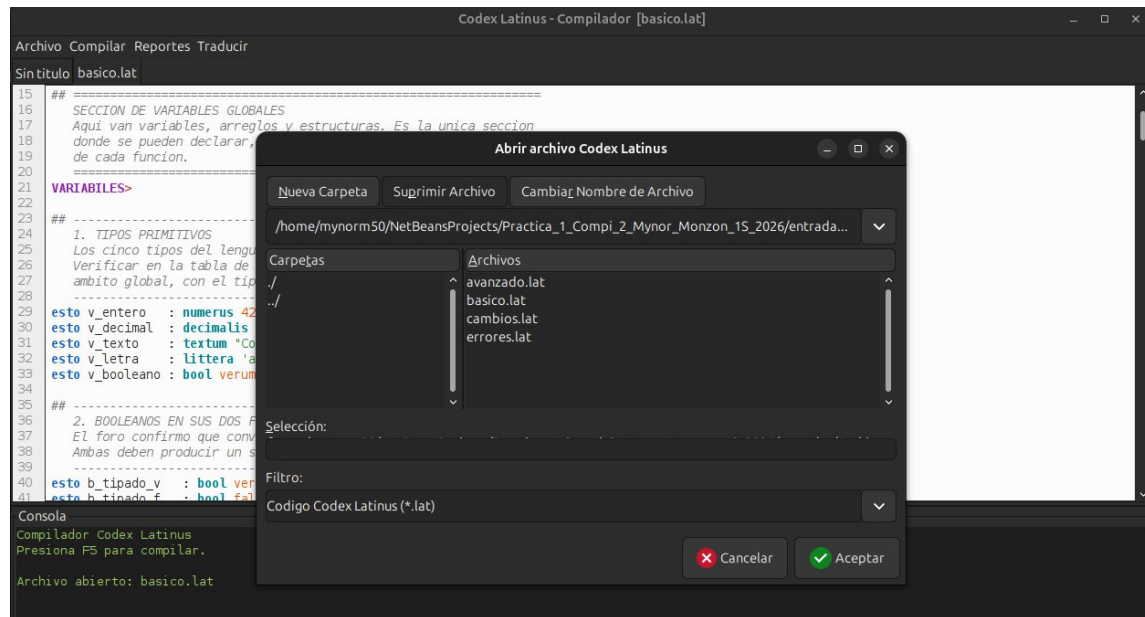
4. Funcionalidades del Sistema

4.1 Crear un archivo nuevo

Ir al menu Archivo y seleccionar Nuevo, o presionar la combinacion Control mas N. Se abre una pestana vacia lista para escribir.

4.2 Abrir un archivo

Ir al menu Archivo y seleccionar Abrir, o presionar Control mas O. Se muestra un selector que filtra los archivos con extension .lat. Al abrir el archivo, el coloreado se aplica de inmediato.



4.3 Guardar un archivo

Ir al menu Archivo y seleccionar Guardar, o presionar Control mas S. Si el archivo ya tiene una ruta asociada, se guarda directamente sobre ella. Si es un archivo nuevo, se solicita la ubicacion.

La opcion Guardar como siempre solicita una ubicacion nueva. Si no se escribe la extension, la aplicacion agrega .lat automaticamente.

4.4 Compilar el codigo

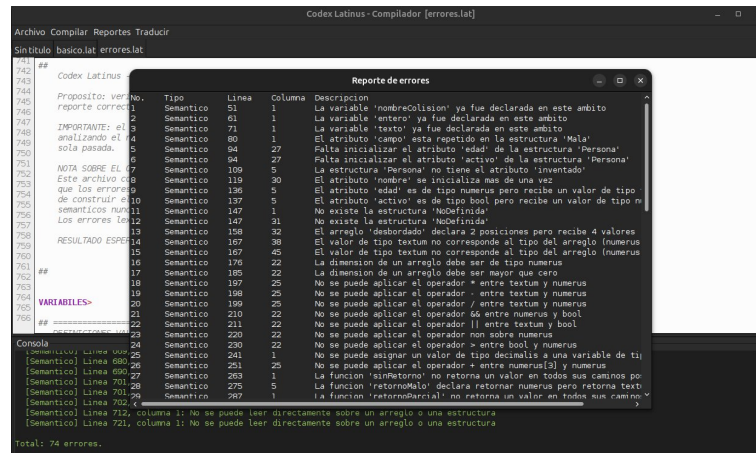
Ir al menu Compilar y seleccionar Compilar, o simplemente presionar la tecla F5. La consola muestra el resultado del analisis.

Si el codigo es correcto, aparece el mensaje de compilacion exitosa y quedan habilitados los reportes y la traduccion. Si se encuentran errores, se listan en la consola con su tipo, linea, columna

4.5 Ver el reporte de errores

Ir al menu Reportes y seleccionar Reporte de errores. Se abre una ventana con la lista completa ordenada por linea, indicando para cada error su numero, tipo, posicion y descripcion.

Los errores se clasifican en tres tipos: lexicos, cuando aparece un caracter que no pertenece al lenguaje; sintacticos, cuando la estructura del codigo no corresponde a ninguna forma valida; y semanticos, cuando la estructura es correcta pero el significado no lo es.

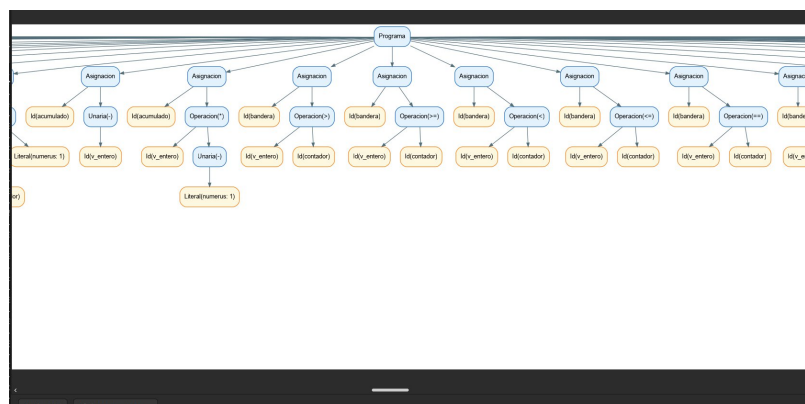


4.6 Graficar el Arbol de Sintaxis Abstracta

Ir al menu Reportes y seleccionar Graficar AST. La aplicacion genera tres archivos en la carpeta reportes del proyecto y abre una ventana con tres pestañas:

- Imagen PNG: muestra el arbol dentro de la aplicacion, con desplazamiento si es muy grande.
- Vectorial SVG: permite abrir la version vectorial en el navegador, que no pierde calidad al ampliarse. Es la opcion recomendada para arboles extensos.
- Codigo DOT: muestra el codigo fuente del grafico, que se puede copiar al portapapeles.

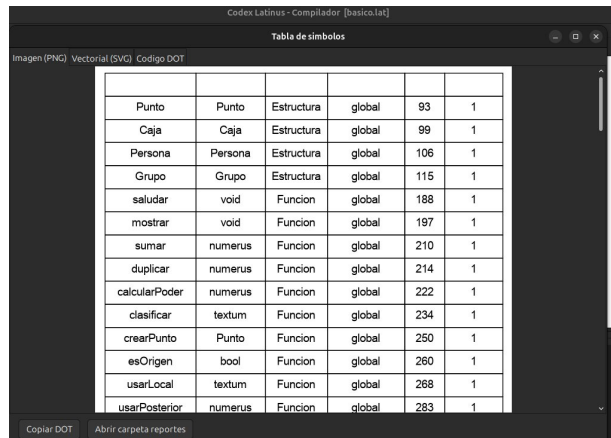
En el grafico, los nodos azules corresponden a nodos internos del arbol y los amarillos a las hojas: literales, identificadores y parametros. Este reporte solo esta disponible despues de compilar un programa sin errores. Si se solicita antes, la aplicacion muestra un aviso indicando que primero se debe compilar.



4.7 Graficar la tabla de simbolos

Ir al menu Reportes y seleccionar Graficar tabla de simbolos. Se muestra una tabla con todos los simbolos declarados durante el analisis, indicando el nombre, el tipo, el rol, el ambito y la posicion de la declaracion.

El ambito indica donde vive cada simbolo. Los declarados en la seccion global aparecen con ambito global; los parametros y variables locales aparecen con el nombre de la funcion; y las variables declaradas en un ciclo per aparecen con el ambito per, porque solo existen dentro de ese ciclo.



Punto	Punto	Estructura	global	93	1
Caja	Caja	Estructura	global	99	1
Persona	Persona	Estructura	global	106	1
Grupo	Grupo	Estructura	global	115	1
saludar	void	Funcion	global	188	1
mostrar	void	Funcion	global	197	1
sumar	numerus	Funcion	global	210	1
duplicar	numerus	Funcion	global	214	1
calcularPoder	numerus	Funcion	global	222	1
clasificar	textum	Funcion	global	234	1
crearPunto	Punto	Funcion	global	250	1
esOrigen	bool	Funcion	global	260	1
usarLocal	textum	Funcion	global	268	1
usarPosterior	numerus	Funcion	global	283	1

4.8 Ver la pila de analisis paso a paso

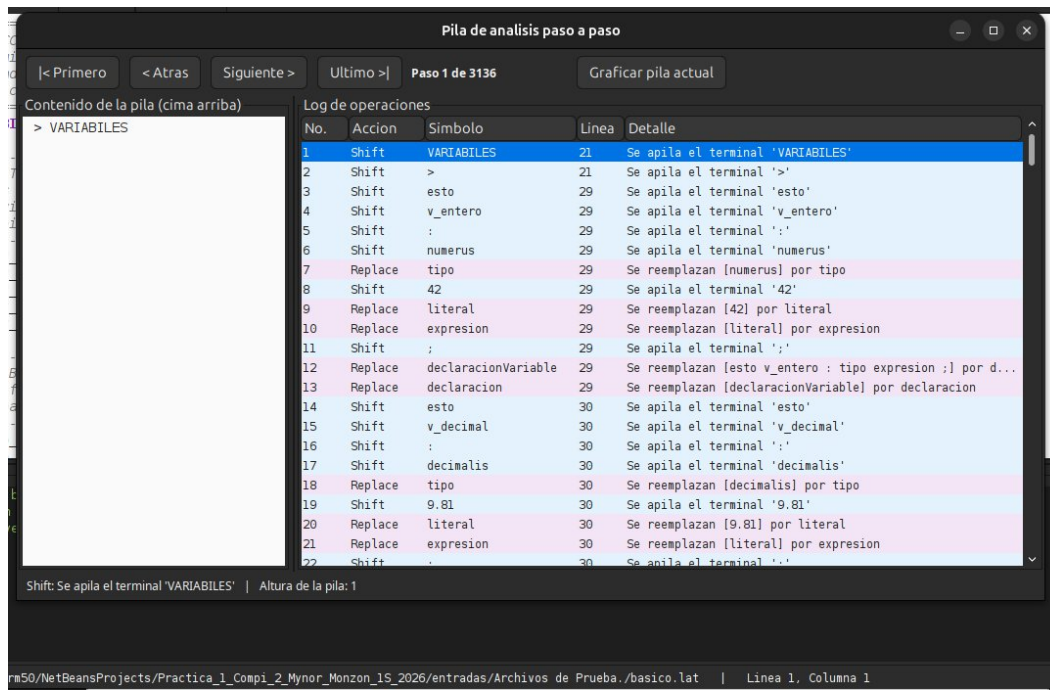
Ir al menu Reportes y seleccionar Pila de analisis paso a paso. Se abre una ventana dividida en dos areas: a la izquierda el contenido de la pila en el paso actual, con la cima en la parte superior; a la derecha el registro completo de operaciones.

La navegacion se realiza con los cuatro botones de la parte superior:

Boton	Accion
Primero	Salta al primer paso del analisis
Atras	Retrocede un paso
Siguiente	Avanza un paso
Ultimo	Salta al ultimo paso, donde se registra la operacion de aceptacion

Tambien se puede saltar directamente a cualquier paso haciendo clic sobre la fila correspondiente del registro. Las filas se colorean segun la operacion: azul para las operaciones de apilado, rosa para las de reemplazo y verde para la aceptacion final.

El boton Graficar pila actual genera una imagen del estado de la pila en el paso seleccionado.

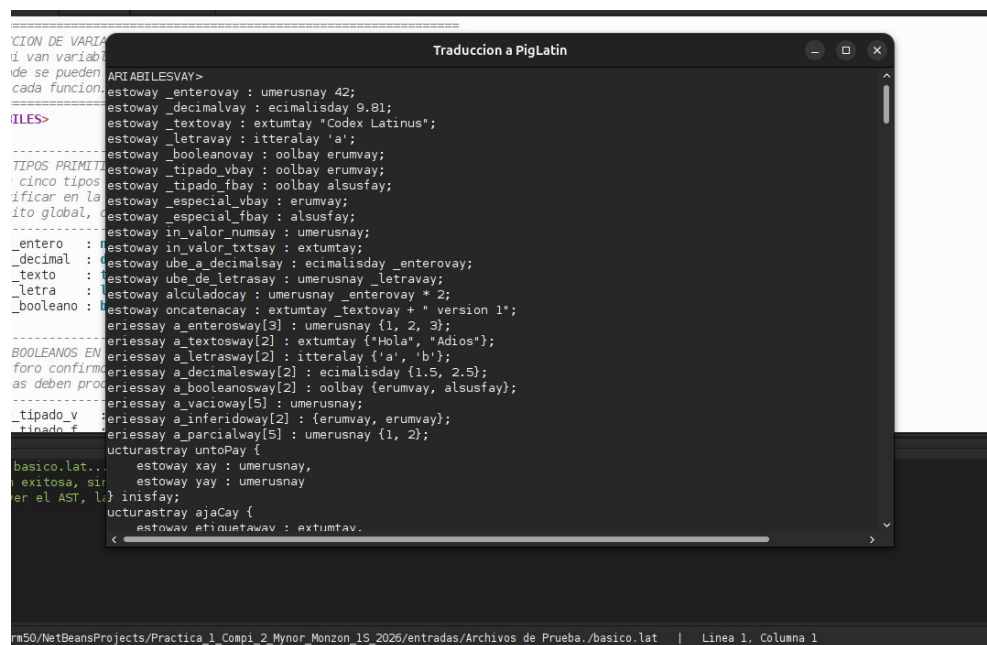


4.9 Ver la traducción a PigLatin

Ir al menú Traducir y seleccionar Ver traducción a PigLatin. Se abre una ventana con el código traducido completo.

La traducción aplica tres leyes. Las palabras que comienzan con consonante trasladan ese grupo inicial al final y agregan el sufijo ay. Las que comienzan con vocal simplemente agregan el sufijo way. Y las funciones especiales de impresión y lectura se sustituyen por sus equivalentes porcinos.

Se traducen los identificadores y las palabras reservadas. Los símbolos del lenguaje, los números y el contenido de las cadenas de texto permanecen sin cambios.



4.10 Descargar la traduccion

Ir al menu Traducir y seleccionar Descargar traduccion. Se solicita la ubicacion donde guardar el archivo, proponiendo como nombre el del archivo fuente pero con extension .pig. Si no se escribe la extension, la aplicacion la agrega automaticamente.

5. Referencia del Lenguaje Codex Latinus

5.1 Estructura de un programa

Todo programa se compone de tres secciones. Las dos primeras son opcionales y la ultima es obligatoria:

```
## Comentario de bloque, puede ocupar varias lineas ##

VARIABLES>
  ## Aqui van las variables, arreglos y estructuras globales ##
  esto edad : numerus 20;

MUNERA>
  ## Aqui van las definiciones de funciones ##
  ratio numerus duplicar(esto valor : numerus) {
    reddere valor * 2;
  } finis;

MAIOR>
  ## Seccion principal, obligatoria ##
  >> "Hola mundo";

FINIS;
```

Notese la diferencia entre FINIS en mayusculas, que cierra el programa completo, y finis en minusculas, que cierra bloques. El lenguaje distingue mayusculas de minusculas.

5.2 Comentarios

Existen dos formas de escribir comentarios:

```
// Comentario de una sola linea

##
  Comentario de bloque
  que puede ocupar varias lineas
##
```

Es importante que los comentarios de bloque esten correctamente balanceados. Una secuencia de dos almohadillas de mas o de menos desplaza la apertura y el cierre de todos los comentarios siguientes, lo que produce errores en lugares alejados del origen real del problema.

5.3 Tipos de datos

Tipo	Descripcion	Ejemplo de valor
numerus	Numero entero	42
decimalis	Numero de punto flotante	9.81
textum	Cadena de caracteres	"Codex Latinus"
littera	Caracter unico	'a'
bool	Valor booleano	verum o falsus

Los tipos se organizan en una jerarquia que determina las conversiones automaticas. De mayor a menor nivel: textum, decimalis, numerus, littera y bool. Un valor puede asignarse a una variable de nivel igual o superior, pero nunca de nivel inferior, porque eso implicaria perder informacion.

5.4 Declaracion de variables

Las variables solo se pueden declarar en la seccion VARIABLES global o en el bloque VARIABLES de una funcion. Nunca dentro de un bloque de control ni en la seccion principal.

```
esto edad : numerus 20;
esto gravedad : decimalis 9.81;
esto nombre : textum "Comandante";
esto inicial : littera 'a';
esto activo : bool verum;

## Forma abreviada para booleanos, sin escribir el tipo ##
esto encendido : verum;
esto apagado : falsus;

## Declaracion sin valor inicial ##
esto contador : numerus;

## Valor calculado a partir de otra variable ##
esto doble : numerus edad * 2;
```

5.5 Arreglos

```
## Con valores iniciales ##
series numeros[3] : numerus {1, 2, 3};
series nombres[2] : textum {"Hola", "Adios"};

## Sin valores iniciales ##
series vacio[5] : numerus;

## Con el tipo inferido de los valores ##
series banderas[2] : {verum, falsus};

## Lectura y escritura por indice ##
numeros[0] = 99;
>> numeros[1];
```

Los indices comienzan en cero. Es valido declarar menos valores iniciales que la dimension, pero nunca mas.

5.6 Estructuras

Los campos de una estructura se pueden separar con punto y coma o con coma, indistintamente. No se permiten valores por defecto en la definicion.

```
estructura Punto {
    esto x : numerus;
    esto y : numerus
} finis;

estructura Persona {
    esto nombre : textum,
    esto edad : numerus,
    esto lugar : Punto
} finis;

## Instanciacion: todos los atributos son obligatorios ##
esto yo : Persona {
    nombre: "Estudiante",
    edad: 22,
    lugar: { x: 1, y: 2 }
}

## Acceso encadenado ##
>> yo.lugar.x;
yo.edad = 23;
```

El orden de los atributos al instanciar no tiene que coincidir con el de la definicion, pero todos deben estar presentes.

5.7 Operadores

Categoria	Operadores	Notas
Aritmeticos	+, -, *, /	El operador + tambien concatena cadenas
Incremento	++, --	Se pueden usar en cualquier parte del codigo
Relacionales	<, >, <=, >=	Producen un valor booleano
Igualdad	==, !=	Producen un valor booleano
Logicos	&&,	Exigen operandos booleanos estrictos
Negacion	non	Solo se aplica sobre booleanos
Agrupacion	()	Altera el orden de evaluacion

La precedencia, de mayor a menor, es: agrupacion, unarios, multiplicacion y division, suma y resta, relacionales, igualdad, conjuncion y finalmente disyuncion.

5.8 Condicionales

```
## Condicional simple ##
si (edad >= 18) {
    >> "Mayor de edad";
} finis;

## Con rama alternativa ##
si (edad >= 18) {
    >> "Mayor de edad";
} aliter {
    >> "Menor de edad";
} finis;

## Cadena de condiciones ##
si (nota >= 90) {
    >> "Excelente";
} aliter (nota >= 70) {
    >> "Bueno";
} aliter {
    >> "Insuficiente";
} finis;
```

La palabra finis aparece una sola vez, al cerrar toda la cadena. Las condiciones deben ser estrictamente booleanas: un numero no se convierte automaticamente.

5.9 Ciclos

```
## Ciclo mientras: la condicion se evalua antes ##
dum (contador < 10) {
    contador++;
} finis;

## Ciclo hacer mientras: el cuerpo corre al menos una vez ##
facere {
    contador++;
} dum (contador < 10);

## Ciclo para ##
per (esto i : numerus 0; i < 10; i++) {
    >> i;
} finis;

## Control de flujo dentro de un ciclo ##
per (esto i : numerus 0; i < 10; i++) {
    si (i == 3) {
        perge; // salta a la siguiente vuelta
    } finis;
    si (i > 7) {
        interrompe; // corta el ciclo
    } finis;
} finis;
```

Las instrucciones perge e interrompe solo se pueden usar dentro de un ciclo. Estar dentro de un condicional no basta si ese condicional no esta a su vez dentro de un ciclo.

5.10 Funciones

```
## Funcion sin retorno ##
actio saludar(esto nombre : textum) {
    >> "Hola " >> nombre;
} finis;

## Funcion con retorno y variables locales ##
ratio numerus calcular(esto fuerza : numerus) {
    VARIABLES[
        esto total : numerus fuerza * 2;
    ]
    reddere total;
} finis;

## Funcion que retorna una estructura ##
ratio Punto crearPunto(esto a : numerus, esto b : numerus) {
    VARIABLES[
        esto p : Punto { x: a, y: b };
    ]
    reddere p;
} finis;

## Llamadas ##
saludar("Mundo");
resultado = calcular(10);
```

Las variables locales de una funcion solo se pueden declarar dentro del bloque VARIABLES, que va al inicio del cuerpo. Una funcion de tipo ratio debe retornar un valor por todos sus caminos posibles.

5.11 Entrada y salida

```
## Impresion de un valor ##
>> "Hola mundo";

## Impresion de varios valores ##
>> "Nombre: " >> nombre >> " Edad: " >> edad;

## Impresion de una expresion ##
>> "Suma: " >> edad + 10;

## Lectura hacia una variable ##
nombre <<

## Lectura hacia un elemento o un atributo ##
numeros[0] <<
persona.edad <<
```

```
## Lectura descartando el valor ##  
<<
```

Ambas funciones especiales se pueden usar en cualquier parte del código: en la sección principal, dentro de una función, dentro de un condicional o dentro de un ciclo.

6. Ejemplo Completo de Uso

A continuación se describe un recorrido completo por la aplicación, desde abrir el programa hasta descargar la traducción.

Paso 1: Abrir el programa

Ejecutar la aplicación. Se abre la ventana principal con una pestaña vacía.

Paso 2: Escribir el código

Escribir o pegar el siguiente programa de ejemplo en el editor. El coloreado se aplica conforme se escribe.

```
##  
    Programa de ejemplo  
##  
VARIABLES>  
esto edad : numerus 20;  
esto cifrado : falsus;  
esto comandante : textum "Estudiante X";  
esto fuerza : numerus 10;  
  
MUNERA>  
ratio numerus calcularPoder(estos fuerza : numerus) {  
    VARIABLES[  
        esto total : numerus fuerza * 2;  
    ]  
    reddere total;  
} finis;  
  
MAIOR>  
>> "Hola comandante!";  
>> "Ingresa tu nombre por favor";  
comandante <<  
>> "Bienvenido" >> comandante;  
  
si (edad >= 18) {  
    cifrado = verum;  
    fuerza = 12;  
} finis;  
  
>> "Tu poder es: " >> calcularPoder(fuerza);  
  
FINIS;
```



Paso 3: Compilar

Presionar F5. La consola muestra el mensaje de compilacion exitosa.

Paso 4: Revisar el AST

Ir a Reportes y seleccionar Graficar AST. Se abre la ventana con el arbol generado.

Paso 5: Revisar la tabla de simbolos

Ir a Reportes y seleccionar Graficar tabla de simbolos. Se muestran las variables globales con ambito global, y el parametro y la variable local de la funcion con ambito calcularPoder.

Paso 6: Recorrer la pila de analisis

Ir a Reportes y seleccionar Pila de analisis paso a paso. Navegar con los botones y observar como se apilan los tokens y como se reemplazan por no terminales al completarse cada regla.

Paso 7: Ver la traduccion

Ir a Traducir y seleccionar Ver traduccion a PigLatin. El codigo aparece transformado:

```

ARIABILESVAY>
estoway edadway : umerusnay 20;
estoway ifradocay : alsusfay;
estoway omandantecay : extumtay "Estudiante X";
estoway uerzafay : umerusnay 10;

UNERAMAY>
atoray umerusnay alcularPodercay(estoway uerzafay : umerusnay) {
    ARIABILESVAY[
        estoway otaltay : umerusnay uerzafay * 2;
    ]
    eddereray otaltay;
} inisfay;

```



```

AIORMAY>
%OINK "Hola comandante!";
omandantecay %OINK_OINK;
isay (edadway >= 18) {
    ifradocay = erumvay;
} inisfay;

```

```

INISFAY;

```

Paso 8: Descargar y guardar

Ir a Traducir y seleccionar Descargar traduccion para obtener el archivo .pig. Luego ir a Archivo y seleccionar Guardar para conservar el codigo fuente con extension .lat.

Codex Latinus - Compilador [Sin titulo]

Arbol de sintaxis abstracta

Imagen (PNG) Vectorial (SVG) Codigo DOT

Codex Latinus - Compilador [Sin titulo]

Tabla de simbolos

Imagen (PNG) Vectorial (SVG) Codigo DOT

calcularPoder	numerus	Funcion	global	11	1
edad	numerus	Variable	global	5	1
cifrado	bool	Variable	global	6	1
comandante	textum	Variable	global	7	1
fuerza	numerus	Variable	global	8	1
fuerza	numerus	Parametro	calcularPoder	11	29
total	numerus	Variable	calcularPoder	13	9

Pila de analisis paso a paso

< Primero < Atras Siguiendo > Ultimo >

Paso 1 de 183 Graficar pila actual

Contenido de la pila (cima arriba)

> VARIABLES

Log de operaciones

No.	Accion	Simbolo	Linea	Detalle
1	Shift	VARIABLES	4	Se apila el terminal 'VARIABLES'
2	Shift	>	4	Se apila el terminal '>'
3	Shift	esto	5	Se apila el terminal 'esto'
4	Shift	edad	5	Se apila el terminal 'edad'
5	Shift	:	5	Se apila el terminal ':'
6	Shift	numerus	5	Se apila el terminal 'numerus'
7	Replace	tipo	5	Se reemplazan (numerus) por tipo
8	Shift	20	5	Se apila el terminal '20'
9	Replace	literal	5	Se reemplazan [20] por literal
10	Replace	expresion	5	Se reemplazan [literal] por expresion
11	Shift	:	5	Se apila el terminal ':'
12	Replace	declaracionVariable	5	Se reemplazan (esto edad : tipo expresion) por declaracionVariable
13	Replace	declaracion	5	Se reemplazan [declaracionVariable] por declaracion
14	Shift	esto	6	Se apila el terminal 'esto'
15	Shift	cifrado	6	Se apila el terminal 'cifrado'
16	Shift	:	6	Se apila el terminal ':'
17	Shift	falsus	6	Se apila el terminal 'falsus'
18	Replace	valorBooleano	6	Se reemplazan [falsus] por valorBooleano
19	Shift	:	6	Se apila el terminal ':'
20	Replace	declaracionVariable	6	Se reemplazan (esto cifrado : valorBooleano) por declaracionVariable
21	Replace	declaracion	6	Se reemplazan [declaracionVariable] por declaracion
22	Shift	esto	7	Se apila el terminal 'esto'

Traduccion a PigLatin

ARIABLESVAY>

estoway edadway : umerusnay 20;

estoway ifradocay : alsusfay;

estoway omandantecay : extumtay "Estudiante X";

estoway uerzafay : umerusnay 10;

AIORMAY>

atoray umerusnay alcularPoderway(estoway uerzafay : umerusnay) {

ARIABLESVAY[

estoway otaltay : umerusnay uerzafay * 2;

]

eddereray otaltay;

inifay;

AIORMAY>

%OINK "Hola comandante!";

%OINK "Ingresa tu nombre por favor";

omandantecay %OINK_OINK;

%OINK "Bienvenido" %OINK omandantecay;

isay (edadway >= 18) {

ifradocay = erumvay;

uerzafay = 12;

inifay;

%OINK "Tu poder es: " %OINK alcularPoderway(uerzafay);

INISFAY;

7. Mensajes de Error y Solucion

La siguiente tabla recoge los mensajes mas frecuentes y como resolverlos.

Mensaje	Causa y solucion
Caracter no reconocido por el lenguaje	Se escribio un simbolo que no pertenece al alfabeto del lenguaje. Revisar la linea indicada y eliminarlo. Tambien puede ocurrir si un comentario de bloque quedo mal balanceado.
La instruccion no corresponde a ninguna forma valida del lenguaje	La estructura escrita no coincide con ninguna regla. Revisar la sintaxis de la construccion en la seccion 5 de este manual.
Falta un simbolo	Se omitio un elemento obligatorio, generalmente un punto y coma, una llave o la palabra finis.
La variable no ha sido declarada	Se usa un nombre que nunca se declaro. Verificar la escritura y que la declaracion este en la seccion VARIABLES.
Ya fue declarada en este ambito	Existen dos declaraciones con el mismo nombre en el mismo nivel. Cambiar uno de los nombres.
Las variables solo se pueden declarar en la seccion VARIABLES	Se intento declarar una variable en la seccion principal o dentro de un bloque de control. Moverla a la seccion VARIABLES.
No se puede aplicar el operador entre dos tipos	La operacion no es valida para esos tipos. Consultar la tabla de operadores de la seccion 5.7.
La condicion debe ser de tipo booleano	La condicion de un si o de un ciclo debe evaluar a verum o falsus. Usar una comparacion en lugar de un numero.
No se puede asignar un valor de un tipo a una variable de otro	La conversion implicita solo opera hacia arriba en la jerarquia. No se puede guardar un decimalis en un numerus.
Falta inicializar el atributo	Al instanciar una estructura deben darse valores a todos sus atributos.
Indice fuera de rango	Se accede a una posicion que no existe. Los indices van desde cero hasta la dimension menos uno.
No retorna un valor en todos sus caminos posibles	Una funcion ratio tiene una ruta de ejecucion que termina sin reddere. Agregar una rama aliter final o un retorno al cierre.
Solo se puede usar dentro de un ciclo	Las instrucciones perge e interrompe requieren estar dentro de un dum, facere o per.
Codigo inalcanzable	Hay instrucciones despues de un reddere que nunca se ejecutarian. Eliminarlas o moverlas antes del retorno.

8. Archivos Soportados

Extension	Uso
.lat	Codigo fuente en lenguaje Codex Latinus. Es el formato que la aplicacion abre y guarda.
.pig	Codigo traducido al idioma PigLatin. Solo se genera, no se abre.
.dot	Codigo fuente de los graficos, generado en la carpeta reportes.
.png	Imagen de los reportes graficos, generada en la carpeta reportes.
.svg	Version vectorial de los reportes graficos, generada en la carpeta reportes.

9. Notas Adicionales

- El compilador analiza y traduce el codigo, pero no lo ejecuta. Las funciones de impresion y lectura se validan y se traducen, pero no producen salida ni piden datos.
- Los reportes graficos y la traduccion solo estan disponibles despues de una compilacion sin errores. Si se solicitan antes, la aplicacion muestra un aviso.
- Los archivos generados se guardan en la carpeta reportes del proyecto y se sobrescriben en cada nueva generacion.
- Se pueden tener varios archivos abiertos al mismo tiempo en pestanas distintas. Cada pestana recuerda su propia ruta.
- El lenguaje distingue mayusculas de minusculas, tanto en las palabras reservadas como en los identificadores.
- Para arboles muy extensos conviene usar la version vectorial del reporte, que permite ampliar sin perder calidad.